

RESEARCH PAPER TO WORKING CODE TO GITHUB

A complete AI-assisted workflow using Paper2Code, Codex or Claude Code

Start with an arXiv paper. End with a clean, tested, reproducible public repository.

The core idea

Do not ask an AI coding agent to “implement this paper” in one giant step. First extract a traceable specification, then build a baseline, verify it against the paper, improve it in controlled passes, and only then publish it.

WORKFLOW AT A GLANCE

1. Pick a paper worth implementing
2. Create a clean project workspace
3. Run Paper2Code to generate a citation-anchored baseline
4. Convert the paper and baseline into an implementation spec
5. Initialize Codex or Claude Code with strict project rules
6. Make the baseline actually run on a tiny test case
7. Verify equations, tensor shapes, losses, hyperparameters, and data flow
8. Improve correctness, code quality, performance, and tests in separate passes
9. Cross-review with a second coding agent
10. Reproduce a small result from the paper
11. Clean the repository and write documentation
12. Publish to GitHub with transparent reproduction notes

1. What Paper2Code adds to this workflow

Paper2Code should be treated as the first-pass implementation generator, not the final authority. The useful open-source paper2code skill accepts an arXiv URL or ID and produces a repository where important implementation decisions are tied back to paper sections and equations. It also flags choices that are unspecified or ambiguous instead of silently pretending the paper gave an answer.

Important naming note

The usable tool in this workflow is the open-source paper2code agent skill by PrathamLearnsToCode. The similarly named research project “Paper2Code” introduces PaperCoder, a planning, analysis, and code-generation pipeline. This guide borrows the best idea from both: structured paper-to-code generation plus aggressive verification.

Paper2Code feature	Why it matters
Citation anchoring	Non-trivial decisions can reference the exact section or equation they implement.
Ambiguity audit	Choices can be classified as SPECIFIED, PARTIALLY SPECIFIED, UNSPECIFIED, or assumptions.
Appendix mining	Appendices, footnotes, figure captions, and implementation details are treated as useful sources.
Repository scaffold	The generated output can include README, reproduction notes, source files, configs, and a walkthrough notebook.
Framework and mode options	You can request a framework and use full or educational modes depending on the goal.

Install the Paper2Code skill

```
npx skills add PrathamLearnsToCode/paper2code/skills/paper2code
```

During installation, select the coding agent you want to use, choose global or project scope, and choose the install method. After that, open the coding agent in your project directory.

Run it on the paper

```
/paper2code https://arxiv.org/abs/XXXX.XXXXX
/paper2code https://arxiv.org/abs/XXXX.XXXXX --mode full
/paper2code https://arxiv.org/abs/XXXX.XXXXX --framework jax
/paper2code https://arxiv.org/abs/XXXX.XXXXX --mode educational
```

For a serious public reproduction, start with full mode if the paper needs a training/data pipeline. Use educational mode when you also want a walkthrough that makes the implementation easier to audit and explain.

2. Pick the right paper and define the goal

Before touching code, decide what “implemented” means. A paper can be reproduced at several levels, and the repository will become much cleaner if you choose one explicitly.

Goal	Definition of done
Core algorithm	Implement the novel method and prove it runs on toy input.
Reference-quality implementation	Implement the method, training loop, data pipeline, evaluation, configs, and tests.
Result reproduction	Attempt to reproduce one or more numbers, plots, or ablations from the paper.
Extension	First reproduce the method, then add one clearly separated improvement or experiment.

Paper selection checklist

- The paper contains a concrete algorithm, architecture, loss, training method, or measurable experiment.
- There is enough detail in the main text plus appendix to infer a working implementation.
- You can access the datasets or create a toy/synthetic substitute.
- The compute requirement is realistic for your goal.
- If official code exists, decide whether you are doing an independent reproduction or an adaptation of the official repository.
- If you inspect or copy official code, check its license before reusing code. Keep attribution clear.

Best first project

Choose a paper with a crisp central contribution and a result that can be sanity-checked cheaply. Avoid beginning with a giant frontier-model training recipe that needs hundreds of GPUs.

3. Create the project workspace

```
mkdir paper-reproduction
cd paper-reproduction
git init
```

Keep the raw paper, generated artifacts, tests, and implementation notes separated. A useful starting structure is:

```
paper-reproduction/
├── paper/
│   ├── paper.pdf
│   └── notes.md
├── src/
├── tests/
├── configs/
├── scripts/
├── notebooks/
├── PAPER_SPEC.md
├── REPRODUCTION_NOTES.md
├── VERIFICATION.md
├── README.md
└── requirements.txt
```

4. First prompt: make the agent understand the paper before improving code

Even after Paper2Code creates a baseline, force the coding agent to re-read the paper and turn it into an explicit implementation contract. This is the most important prompt in the workflow.

You are reproducing a research paper, not inventing a similar system.

Read the complete paper, including appendix, equations, algorithms, tables, figure captions, and implementation details. Then inspect the current repository generated from the paper.

Create PAPER_SPEC.md with:

1. The paper's central contribution in 5-10 bullets.
2. Every module that must exist in code.
3. Inputs, outputs, tensor shapes, and data types for each module.
4. Every equation that directly affects implementation, translated into pseudocode.
5. Architecture details and execution order.
6. Loss functions and objective terms.
7. Training procedure and optimizer/scheduler settings.
8. Dataset preprocessing and evaluation procedure.
9. Every stated hyperparameter with the exact paper section/table/equation where it appears.
10. A separate ambiguity table with columns: decision, status, evidence, chosen value, alternatives.

Use these statuses only:

- SPECIFIED
- PARTIALLY_SPECIFIED
- UNSPECIFIED
- ASSUMPTION
- FROM_OFFICIAL_CODE

Do not modify code yet. Do not silently fill missing details. Quote or cite the paper section/equation for every important decision.

5. Initialize Codex or Claude Code with project rules

Give the coding agent persistent repository-level instructions so the same standards apply to every later task.

If using Codex

Use ``/init`` to generate an ``AGENTS.md`` scaffold for the project, then edit it with the rules below.

```
/init
```

If using Claude Code

Use ``/init`` to generate or improve the project ``CLAUDE.md``, then add the same rules. Claude Code also supports reusable skills, so Paper2Code can be invoked directly once installed.

```
/init
```

Put these rules in AGENTS.md or CLAUDE.md

```
# Research Reproduction Rules
```

1. PAPER_SPEC.md is the implementation contract.
2. The research paper is the source of truth.
3. Never silently invent a hyperparameter or algorithmic detail.
4. Mark unresolved choices as [UNSPECIFIED], [PARTIALLY_SPECIFIED], or [ASSUMPTION].
5. Keep important paper section/equation references near the corresponding code.
6. Before changing behavior, explain which paper requirement the change satisfies.
7. Keep functions small and testable.
8. Add tests for tensor shapes, invariants, losses, and edge cases.
9. Run the smallest relevant test after every meaningful change.
10. Do not optimize performance until correctness tests pass.
11. Do not claim a paper result is reproduced unless the exact experiment was actually run.
12. Maintain REPRODUCTION_NOTES.md with deviations from the paper.
13. Maintain VERIFICATION.md with checks performed and their status.

6. Make the Paper2Code baseline actually run

The first engineering milestone is not “match the paper.” It is “the smallest faithful implementation executes end to end.” Keep the input tiny so failures are cheap to diagnose.

Inspect PAPER_SPEC.md, REPRODUCTION_NOTES.md, configs, and the generated source code.

Your task is to make the smallest end-to-end execution succeed without changing the intended algorithm.

Do this in order:

1. Identify import, dependency, API, shape, dtype, and device errors.
2. Create a tiny synthetic input or tiny dataset fixture.
3. Make one forward pass execute.
4. Make the loss compute.
5. If training is in scope, make one optimizer step execute.
6. Add a smoke test that reproduces this path.
7. Run the test and show the result.

For every fix, classify it as:

- implementation bug
- environment/API compatibility fix
- paper ambiguity
- intentional deviation

Do not refactor for style yet. Correctness and executability only.

Minimum smoke-test targets

- Imports succeed in a fresh environment.
- Model or algorithm object can be constructed from config.
- A tiny input passes through the core method.
- Output shapes and dtypes are plausible.
- Loss is finite when applicable.
- Backward pass and one optimizer step work when training is in scope.
- CPU execution works for a toy case when realistically possible.

7. Build a paper-to-code verification matrix

Now verify fidelity. Ask the agent to inspect the implementation against the paper one requirement at a time instead of trusting that runnable code is correct.

Audit the repository against PAPER_SPEC.md and the original paper.

Create or update VERIFICATION.md as a table with:

- Requirement
- Paper reference
- Code location
- Test that verifies it
- Status: PASS / FAIL / UNVERIFIED
- Notes

Audit at minimum:

1. equations and mathematical operations
2. tensor dimensions and broadcasting
3. ordering of layers/operations
4. normalization placement
5. residual connections
6. masking and boundary conditions
7. loss terms and weighting
8. initialization
9. optimizer and scheduler
10. data preprocessing
11. augmentation
12. train/eval behavior
13. metrics
14. random seeds and determinism
15. all hyperparameters

Do not fix anything yet. Produce the audit first, sorted by severity.

Then fix only the highest-severity fidelity issues

Take the FAIL items from VERIFICATION.md in severity order.

For each item:

1. Explain the mismatch.
2. Cite the paper section/equation.
3. Propose the smallest faithful code change.
4. Add or update a test that would fail before the fix.
5. Apply the fix.
6. Run the relevant tests.
7. Update VERIFICATION.md.

Handle one conceptual issue at a time. Avoid unrelated refactors.

8. Improve the implementation in separate passes

Do not mix correctness, cleanup, speed, and documentation into one large agent task. Separate passes make regressions much easier to detect.

Pass A: correctness

Remove remaining fidelity failures, shape bugs, incorrect defaults, and hidden assumptions.

Pass B: tests

Add unit, invariant, gradient, numerical, and smoke tests.

Pass C: architecture

Reduce duplication, improve module boundaries, type hints, configuration, and dependency management without changing behavior.

Pass D: performance

Profile first, then optimize bottlenecks while checking numerical equivalence.

Pass E: usability

Add CLI entry points, example configs, reproducible commands, and useful error messages.

Pass F: documentation

Explain what is implemented, what is not reproduced, and which choices differ from the paper.

Prompt: test hardening

Act as a research-software test engineer.

Do not add features. Expand the test suite so that incorrect implementations of the paper are likely to be caught.

Add tests for:

- expected tensor shapes
- mathematical invariants
- edge cases and masks
- deterministic behavior under a fixed seed where appropriate
- finite outputs/losses
- gradient flow
- serialization/checkpoint round trips if relevant
- one minimal end-to-end training/evaluation path

Every test should state which paper requirement or implementation invariant it protects. Run the full suite and report failures.

Prompt: safe refactor

Refactor this repository for readability and maintainability without changing algorithmic behavior.

Constraints:

- keep all current tests passing
- preserve paper equation/section references
- do not change numerical defaults
- do not silently resolve ambiguous choices
- keep public interfaces stable unless there is a strong reason

Prioritize:

1. remove duplication
2. improve names while retaining mapping to paper notation
3. centralize configuration
4. add type hints/docstrings
5. isolate data/model/loss/evaluation responsibilities

After each group of changes, run tests.

Prompt: performance pass

Profile the implementation before optimizing it.

Identify the top runtime and memory bottlenecks on a representative small workload. Then propose optimizations that preserve the paper's semantics.

For each optimization:

- state expected benefit
- state correctness risk
- add a numerical-equivalence or tolerance test where appropriate
- benchmark before and after

Do not change the algorithm merely to make it faster. Keep optimization commits separate from correctness commits.

9. Cross-review with a second coding agent

A strong workflow is to let one agent implement and a second agent attack the implementation. For example, generate and improve with Claude Code, then ask Codex to review, or do the reverse. The second agent should not be told to rewrite everything. It should produce a defect list first.

You are an independent reviewer of a research-paper reproduction.

Assume the current implementation may be subtly wrong even if tests pass. Read:

- the paper
- PAPER_SPEC.md
- REPRODUCTION_NOTES.md
- VERIFICATION.md
- source code
- tests

Find discrepancies between the paper and code. Focus on errors that can change experimental results.

Return findings sorted by severity:
CRITICAL, HIGH, MEDIUM, LOW.

For each finding include:

1. paper reference
2. code location
3. why it is inconsistent or risky
4. a concrete test that would expose it
5. minimal recommended fix

Do not edit files until the review is complete.

After the review, give the implementation agent only the accepted findings and ask it to fix them one at a time with regression tests.

10. Reproduce a small paper result

Before publishing, try to reproduce at least one cheap and meaningful result if the paper allows it. This could be a table row on a small dataset, a qualitative output, a known loss trend, an ablation direction, or a toy numerical example.

Select the cheapest experiment that meaningfully validates the implementation against the paper.

Create scripts/reproduce_minimal.py and document the exact command.

The experiment must:

1. state which paper result it targets
2. use a pinned config
3. log seed and environment
4. save raw metrics
5. compare observed vs reported result
6. clearly label the outcome as MATCH, PARTIAL MATCH, or NOT REPRODUCED

Do not overclaim. If the compute/data/setup differs from the paper, document the difference.

11. Freeze reproducibility artifacts

- Pin dependencies using a lockfile or pinned environment file.
- Commit default and reproduction configs.
- Record Python/framework/CUDA versions when relevant.
- Provide deterministic seeds where meaningful.
- Keep a command that runs the smoke test from a clean checkout.
- Keep a command that runs the minimal reproduction experiment.
- Commit sample output or expected metrics when small enough.
- Keep large datasets and checkpoints out of Git; document how to obtain them.

Prompt: environment reproducibility audit

Audit this repo as if a stranger will clone it on a clean machine.

Find anything that depends on my local environment, hidden files, absolute paths, unpinned versions, undeclared packages, unavailable data, missing environment variables, or manual setup.

Fix safe issues and create a CLEAN_SETUP_CHECKLIST.md with exact steps from clone to smoke test. Do not add secrets or machine-specific paths.

12. Create a GitHub-quality README

The README should make the repository useful even to someone who never saw your development process. It should distinguish paper claims, your implementation, and actual reproduced results.

Write a polished README.md for this independent research-paper implementation.

Use this structure:

1. Project title
2. One-sentence description
3. Paper citation and link
4. What this repository implements
5. What it does NOT implement
6. Architecture / algorithm overview
7. Repository structure
8. Installation
9. 60-second quick start
10. Minimal example
11. Training command if applicable
12. Evaluation command if applicable
13. Reproduction status table
14. Known deviations and assumptions
15. Tests
16. Results
17. Limitations
18. Citation
19. License

Important:

- Never imply this is the authors' official repository unless it is.
- Do not claim results that were not actually reproduced.
- Link REPRODUCTION_NOTES.md and VERIFICATION.md prominently.
- Keep commands copy-pasteable.

Recommended reproduction status table

Paper result	Paper value	Our value	Status
Toy forward pass	N/A	Pass	Verified
Reported metric X	0.xx	0.xx or not run	Match / Partial / Not reproduced
Ablation Y	direction / value	observed	Match / Partial / Not reproduced

13. Final repository cleanup before GitHub

```
git status
git diff
pytest -q # or the project's test command
```

Final agent audit prompt

Prepare this repository for public GitHub release.

Do a final read-only audit first. Check for:

- secrets, API keys, tokens, private paths, emails, internal hostnames
- generated junk files and large binaries
- broken setup instructions
- missing dependencies
- stale comments and TODOs
- accidental claims that results were reproduced when they were not
- missing paper citation
- missing license/attribution notes
- inconsistent configs
- failing tests
- references to files that do not exist

Then give me a release checklist. Only after the checklist, make safe cleanup changes and rerun the tests.

Suggested .gitignore

```
.venv/
__pycache__/
*.pyc
.env
.env.*
.DS_Store
.pytest_cache/
.mypy_cache/
wandb/
runs/
checkpoints/
data/
*.ckpt
*.pt
*.pth
```

Publish

```
git add .
git commit -m "Initial paper reproduction"
git branch -M main
git remote add origin <YOUR_GITHUB_REPO_URL>
git push -u origin main
```

If you use GitHub CLI, you can instead create and push the repository from the terminal. Review the repository visibility, license, and files before the first public push.

14. The complete end-to-end sequence

1. Choose a paper and write down the exact reproduction goal.
2. Create a fresh Git repository and store the paper/notes locally.
3. Install the paper2code skill.
4. Run `/paper2code <arXiv URL> --mode full` or the most appropriate mode.
5. Inspect the generated `REPRODUCTION_NOTES.md` before trusting any code.
6. Ask Codex or Claude Code to create `PAPER_SPEC.md` from the full paper.
7. Run `/init` and put the reproduction rules into `AGENTS.md` or `CLAUDE.md`.
8. Make the smallest end-to-end path execute using toy data.
9. Add a smoke test and commit this baseline.
10. Create `VERIFICATION.md` and audit every paper requirement against code.
11. Fix fidelity failures one at a time, each with a regression test.
12. Run a separate test-hardening pass.
13. Run a separate code-quality refactor pass.
14. Profile and optimize only after correctness is stable.
15. Have a second coding agent independently review the repository.
16. Fix accepted review findings with tests.
17. Attempt one minimal reproduction experiment.
18. Freeze configs, environment, seeds, and exact commands.
19. Generate the README and clearly state deviations and reproduction status.
20. Run the public-release audit, remove secrets/junk, rerun tests.
21. Push to GitHub and tag the first reproducible release.

15. Master prompt for an existing Paper2Code repository

If you already ran Paper2Code and want one prompt that kicks off the disciplined workflow, use this. It intentionally tells the agent not to code immediately.

I generated this repository from a research paper using Paper2Code. Your job is to turn the generated baseline into a faithful, tested, public-quality independent reproduction.

Do NOT start by rewriting code.

Phase 1: Understand

- Read the complete paper and appendix.
- Read all repository files.
- Create PAPER_SPEC.md with modules, equations, shapes, losses, hyperparameters, training/data/eval procedure, and paper references.
- Create an ambiguity table using SPECIFIED / PARTIALLY_SPECIFIED / UNSPECIFIED / ASSUMPTION / FROM_OFFICIAL_CODE.

Phase 2: Baseline

- Make the smallest end-to-end path execute with toy data.
- Add a smoke test.
- Classify every required fix.

Phase 3: Verify

- Create VERIFICATION.md mapping paper requirements to code locations and tests.
- Mark each PASS / FAIL / UNVERIFIED.
- Do not fix issues until the audit is complete.

Phase 4: Correct

- Fix FAIL items by severity, one conceptual issue at a time.
- Add a regression test for each important fix.

Phase 5: Harden

- Expand tests for shapes, invariants, gradients, edge cases, determinism, and end-to-end execution.
- Refactor without changing behavior.
- Profile and optimize only if needed.

Phase 6: Reproduce

- Identify the cheapest meaningful paper result to test.
- Add an exact reproduction script/config.
- Report MATCH / PARTIAL MATCH / NOT REPRODUCED without overclaiming.

Phase 7: Release

- Audit setup from a clean machine perspective.
- Write a GitHub-quality README.
- Update REPRODUCTION_NOTES.md.
- Check for secrets and local paths.
- Run the full test suite.

At the end of every phase, stop and summarize: files changed, commands run, tests/results, remaining risks. Keep paper fidelity above code elegance.

16. Advanced: use both agents as a builder/reviewer pair

For higher confidence, give the two agents different roles rather than letting both edit simultaneously.

Role	Tool	Responsibility
Builder	Paper2Code + Claude Code or Codex	Implements one milestone, writes tests, updates notes.
Reviewer	The other coding agent	Performs read-only paper-to-code audit and proposes defects/tests.
Builder	Original builder	Accepts/rejects findings, fixes accepted issues, reruns tests.
Reviewer	Second agent	Rechecks only changed areas and remaining high-risk assumptions.

Why this works

The builder optimizes for progress. The reviewer optimizes for finding mismatches. Keeping these objectives separate is more useful than asking a single agent to repeatedly tell itself that its own code is correct.

17. What a strong final GitHub repository should contain

- README.md with clear independent-implementation wording.
- PAPER_SPEC.md linking the paper to implementation requirements.
- REPRODUCTION_NOTES.md documenting ambiguity, assumptions, and deviations.
- VERIFICATION.md mapping claims to code and tests.
- Pinned environment/dependencies.
- Configs for quick-start and reproduction experiments.
- Core source code with paper references on non-obvious decisions.
- Tests that protect mathematical and structural invariants.
- A minimal runnable example.
- At least one explicit reproduction status, even if the honest status is NOT REPRODUCED.
- Paper citation and appropriate license/attribution information.

Release gate

Do not publish as “reproduced” unless these are true

The code runs from a clean setup, tests pass, critical paper requirements are verified, ambiguous decisions are documented, and any claimed result was actually executed under a documented configuration.